Warsaw apartment prices analysis dashboard

Tymoteusz Pawełczyk

Project overview

- Interactive analytics platform for the **Warsaw apartment market**
- Explores **sale prices, rental prices, and yield dynamics**
- Advanced filtering by location, district, price, size, building features
- Distributional analysis (medians, percentiles, outliers) instead of simple averages
- District-level comparisons and sub-market analysis
- Time-series tracking of prices, rents, and yields
- Built as a reproducible data pipeline with a scalable dashboard frontend

Dataset overview – Warsaw apartments

Dataset description: apartment sale and rent listings for the largest Polish cities, created and maintained by Krzysztof Jamróz, sourced from Kaggle and enriched with OpenStreetMap data; collected monthly between August 2023 and June 2024.
Link to the dataset: https://www.kaggle.com/datasets/krzysztofjamroz/apartment-prices-in-poland/data

- **Identification & time**: listing ID, property fingerprint, month
- **Location**: latitude, longitude, district, distance to city centre
- **Neighborhood**: POI count, distance to schools, kindergartens, universities, clinics, pharmacies, restaurants, post offices
- **Apartment attributes**: size (m²), rooms, floor, total floors, build year
- **Building & legal**: building material, ownership type, apartment condition
- **Amenities**: balcony, parking space, elevator, security, storage room
- **Market & pricing**: sale vs rent, primary vs secondary market, total price, price per m²

Districts analysis data

- Used spatial polygons of Warsaw districts to assign each apartment listing to a district
- District boundaries sourced in **GeoJSON format** from open GitHub repository: https://github.com/andilabs/warszawa-dzielnice-geojson GitHub
- Performed **point-in-polygon spatial join** (each listing's lat/lon matched to nearest district polygon)
- Implemented using **GeoPandas + Shapely** in Python
- Result: new district column linking each listing to its administrative district

Architecture and techstack

Architecture

```
├── data/
│   ├── raw/              # Raw CSV data
│   ├── processed/        # apartments.duckdb
│   └── reference/        # warsaw_districts.geojson
├── src/apartments/       # Main Python package
│   ├── cleaning.py       # Data cleaning
│   ├── fingerprint.py    # Deduplication
│   ├── io.py             # Data loading
│   ├── labels.py         # Label mappings
│   ├── location.py       # Geocoding
│   ├── rental_yield.py   # Yield calculations
│   └── viz.py            # Plotly visualizations
├── streamlit_app/        # Streamlit dashboard
│   ├── app.py            # Main app
│   ├── components/       # UI components
│   │   ├── loaders.py    # DuckDB loaders
│   │   ├── sidebar.py    # Filters
│   │   └── ui.py         # CSS and components
│   └── pages/            # Dashboard pages
│       ├── 1_Sale.py
│       ├── 2_Rent.py
│       ├── 3_Yield.py
│       └── 4_Time_Series.py
├── scripts/              # Processing scripts
│   ├── build_db.py       # Database build
│   └── build_dataset.py  # Data preparation
├── tests/
├── environment.yml       # Conda environment
├── setup.py              # Package installation
└── README.md
```

Techstack

Programming & core
- Python
- pandas, NumPy, SciPy

Data storage & querying
- DuckDB
- SQLAlchemy
- PostgreSQL (psycopg2)

Visualization & analytics
- Plotly
- Matplotlib

Geospatial processing
- GeoPandas

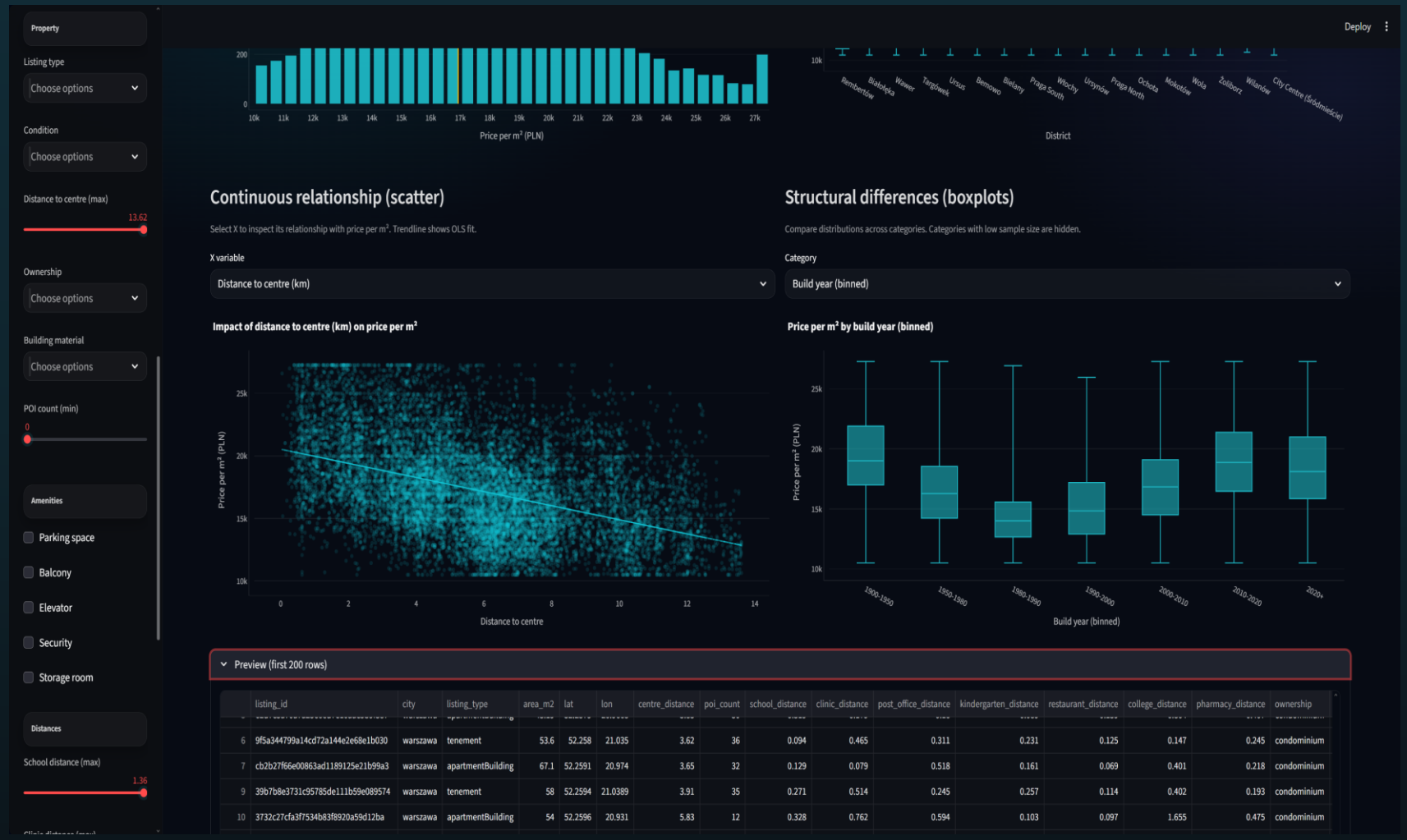
Web & dashboard
- Streamlit

Sale page 1/2

Sale analytics – key features

- KPI overview: number of listings, median price per m², median price, average area
- Interactive filters: location, district, price, price per m², area, floor, building features
- Price per m² distribution with outlier clipping
- District-level price comparison (boxplots)
- Relationship analysis (price per m² vs apartment featurues)
- Structural comparisons by featurues
- Fully interactive, real-time recomputation

Sale page 2/2

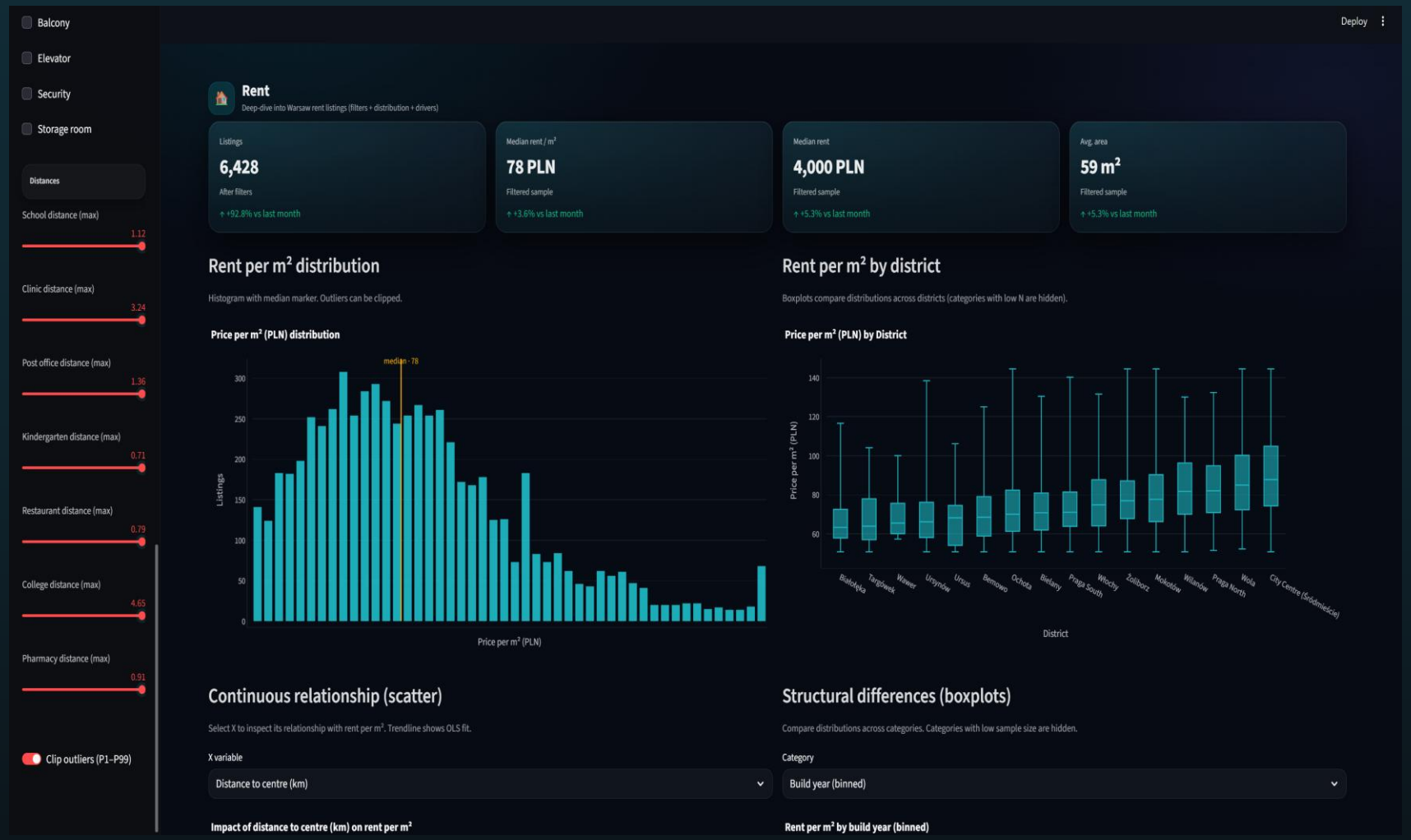


	listing_id	city	listing_type	area_m2	lat	lon	centre_distance	poi_count	school_distance	clinic_distance	post_office_distance	kindergarten_distance	restaurant_distance	college_distance	pharmacy_distance	ownership
6	9f5a344799a14cd72a144e2e68e1b030	warszawa	tenement	53.6	52.258	21.035	3.62	36	0.094	0.465	0.311	0.231	0.125	0.147	0.245	condominium
7	cb2b27f66e00863ad1189125e21b99a3	warszawa	apartmentBuilding	67.1	52.2591	20.974	3.65	32	0.129	0.079	0.518	0.161	0.069	0.401	0.218	condominium
9	39b7b8e3731c95785de111b59e089574	warszawa	tenement	58	52.2594	21.0389	3.91	35	0.271	0.514	0.245	0.257	0.114	0.402	0.193	condominium
10	3732c27cfa3f7534b83f8920a59d12ba	warszawa	apartmentBuilding	54	52.2596	20.931	5.83	12	0.328	0.762	0.594	0.103	0.097	1.655	0.475	condominium

Sale analytics – key features

- KPI overview: number of listings, median price per m², median price, average area
- Interactive filters: location, district, price, price per m², area, floor, building features
- Price per m² distribution with outlier clipping
- District-level price comparison (boxplots)
- Relationship analysis (price per m² vs apartment featurues)
- Structural comparisons by featurues
- Fully interactive, real-time recomputation

Rent page



Rent analytics – key features

- KPI overview: number of listings, median rent per m², median rent, average area
- Interactive filters: location, district, price, rent per m², area, floor, building features
- Rent per m² distribution with outlier clipping
- District-level rent comparison (boxplots)
- Relationship analysis (rent per m² vs apartment featurues)
- Structural comparisons by featurues
- Fully interactive, real-time recomputation

Yield page

Yield analytics – key features

- KPI overview: number of rental listings, median gross yield, number of groups
- Gross yield proxy calculated from rent-to-price relationships
- District-level yield comparison
- Flexible grouping (district or other categories)
- Interactive filters aligned with sale and rent views
- Tabular summary with median, percentiles, and averages

Time series page 1/2

Time series analytics – key features

- KPI overview: number of months, latest median sale price per m², latest yield proxy
- Monthly trends for sale prices (median price per m²)
- Monthly trends for rental prices (median rent per m²)
- Gross yield proxy over time (rent-to-price ratio)
- Market dynamics analysis combining prices, rents, and yields

Time series page 2/2

Time series analytics – key features

- KPI overview: number of months, latest median sale price per m², latest yield proxy
- Monthly trends for sale prices (median price per m²)
- Monthly trends for rental prices (median rent per m²)
- Gross yield proxy over time (rent-to-price ratio)
- Market dynamics analysis combining prices, rents, and yields

APP link

https://warsaw-apartments-analysis.streamlit.app

Code elements – assigning districts

```python
def assign_district_warsaw(
    df: pd.DataFrame,
    *,
    districts_path: Path,
    districts_name_col: str,
    out_col: str = "district",
    lat_col: str = "lat",
    lon_col: str = "lon",
    keep_outside: bool = True,
) -> pd.DataFrame:
    """
    Assign Warsaw district using point-in-polygon join.
    """
    import geopandas as gpd

    missing = [c for c in (lat_col, lon_col) if c not in df.columns]
    if missing:
        raise ValueError(f"Missing required columns: {missing}")

    x = df.dropna(subset=[lat_col, lon_col]).copy()

    points = gpd.GeoDataFrame(
        x,
        geometry=gpd.points_from_xy(x[lon_col], x[lat_col]),
        crs="EPSG:4326",
    )

    districts = gpd.read_file(districts_path)
    districts = districts[districts[districts_name_col].astype("string").str.lower() != "warszawa"].copy()

```

```python
    if districts.crs is None:
        raise ValueError("Districts file has no CRS.")
    districts = districts.to_crs(points.crs)

    if districts_name_col not in districts.columns:
        raise ValueError(
            f"Column '{districts_name_col}' not found in districts file. "
            f"Available: {list(districts.columns)}"
        )

    joined = gpd.sjoin(
        points,
        districts[[districts_name_col, "geometry"]],
        how="left",
        predicate="within",
    )

    joined[out_col] = (
    joined[districts_name_col]
        .astype("string")
        .str.strip()
        .str.lower()
        .str.replace(r"\s+", "_", regex=True)
    )



    out = joined.drop(
        columns=["geometry", "index_right", districts_name_col],
        errors="ignore",
    )

    if not keep_outside:
        out = out.dropna(subset=[out_col]).copy()

    return pd.DataFrame(out)
```

Code elements – property fingerprints

(for month over month duplicates elimination)

```python
@dataclass(frozen=True)
class FingerprintConfig:
    """
    Configuration for property fingerprint generation.

    Attributes:
        lat_round: Decimal places for latitude rounding (default: 4)
        lon_round: Decimal places for longitude rounding (default: 4)
        area_step: Quantization step for area in m² (default: 1.0)
        fingerprint_cols: Additional stable columns to include

    Note:
        Rounding reduces sensitivity to coordinate jitter and measurement variance.
    """
    lat_round: int = 4
    lon_round: int = 4
    area_step: float = 1.0

    fingerprint_cols: Tuple[str, ...] = (
        "market",
        "city",
        "listing_type",
        "build_year",
        "floors_total",
        "floor",
        "has_elevator",
        "has_balcony",
        "has_parking_space",
        "has_security",
        "has_storage_room",
    )


def _quantize(series: pd.Series, step: float) -> pd.Series:
    """
    Quantize numeric values to fixed grid.

    Args:
        series: Numeric series to quantize
        step: Grid size (must be > 0)

    Returns:
        Quantized series aligned to grid
    """
    if step <= 0:
        raise ValueError("area_step must be > 0")
    return (series / step).round(0) * step
```

```python
def _normalize_for_key(series: pd.Series) -> pd.Series:
    """
    Convert column to deterministic string representation.

    Handles missing values with stable tokens:
    - Boolean: False
    - Numeric: -999999
    - Other: "<NA>"

    Args:
        series: Column to normalize

    Returns:
        String series suitable for fingerprint concatenation
    """
    if pd.api.types.is_bool_dtype(series) or str(series.dtype) == "boolean":
        return series.fillna(False).astype(int).astype(str)

    if pd.api.types.is_numeric_dtype(series):
        return series.fillna(-999999).astype(str)

    return series.fillna("<NA>").astype(str)
```

Code elements – yield by category

```python
def make_yield_df(
    df_sale_static: pd.DataFrame,
    df_rent: pd.DataFrame,
    category: str,
    *,
    rent_ppm2_col: str = "price_per_m2",
    sale_ppm2_col: str = "price_per_m2",
    annualization: float = 12.0,
    yield_col: str = "gross_yield_pct",
    benchmark_col: str = "sale_ppm2_benchmark",
) -> pd.DataFrame:
    """
    Create a rent-level yield proxy using a SALE benchmark aggregated by `category`.

    For each rent listing:
        yield_pct = (rent_ppm2 * annualization / median(sale_ppm2 | category)) * 100
    """
    # Compute median sale price per m² benchmark for each category
    bench = (
        df_sale_static
        .groupby(category)[sale_ppm2_col]
        .median()
        .rename(benchmark_col)
        .reset_index()
    )

    # Merge benchmark with rental data and calculate yield
    out = df_rent.merge(bench, on=category, how="inner")
    out[yield_col] = (out[rent_ppm2_col] * annualization / out[benchmark_col]) * 100
    return out
```

THANK YOU FOR YOUR ATTENTION